
Inspectable Symbolic Compression for Long-Context Reasoning: A Mixed-Results Study Against Natural-Language Summaries

Taehyuk Kwon
gtpk11@gmail.com

Abstract

Prior work on reducing the cost of long-context reasoning in large language models (LLMs) either selects salient tokens from the source or compresses the context into continuous vectors. We study *Inspectable Symbolic Compression* (ISM), which converts a long document into reusable discrete text symbols (Z1, Z2, ...) with a short dictionary, and evaluate it honestly against a same-model natural-language summary (Model Summary) at a matched token budget. We report **mixed results**. (RQ1) The internal structure of ISM is genuinely used by the reasoner: at dev scale, inverting the dictionary’s conclusions and removing the symbolic structure each reduce accuracy significantly, while permuting the arbitrary labels does not. (RQ3) However, at a matched budget Model Summary significantly outperforms ISM in both accuracy and efficiency. (RQ4) On the reuse cost-accuracy frontier, Model Summary also dominates ISM. Thus *prompt-only ISM is not a more efficient compressor than a natural-language summary*. We additionally show that an early null result (“corrupting the dictionary does not hurt accuracy”) was an artifact of low compression purity and a content-preserving corruption design rather than evidence against the hypothesis; after correcting both, the RQ1 signal appears. We do not claim ISM as a better compressor. Instead, our findings delimit the limits of prompt-only symbolic compression and argue that future work should move from text that an LLM re-reads to a *programmatically executable semantic intermediate representation* handled by a parser/checker/executor. Label-swap (RQ2) and full registered-scale runs remain future work.

Keywords: long-context reasoning, prompt compression, discrete representation, intervenability, large language models.

1 Introduction

LLMs answer complex queries over long documents, retrieval results, and dialogue history, but reasoning cost and latency grow with input length. Selective Context and the LLMLingua family remove low-importance text [10, 7, 8, 9]; Gist Tokens, AutoCompressors, and ICAE compress context into a few continuous memory representations [11, 12, 13]. These methods retain strong performance at high compression ratios, but their evaluation largely focuses on accuracy and token count before and after compression: what information units a compressed representation holds, and how reasoning changes when those units are altered, is comparatively under-studied.

We study *Inspectable Symbolic Compression* (ISM), which converts a long document into short discrete text symbols with a dictionary, e.g. a conditional rule rendered as **Z1: marker_a high & marker_b low -> risk+**. Because the representation is made of ordinary tokens with an explicit symbol–meaning mapping, one can remove the dictionary, corrupt its mappings, or swap labels in a controlled way and observe the effect on downstream reasoning.

Our starting hypothesis was that ISM could be a *competitive-efficiency yet manipulable* compressed representation. At dev scale this efficiency claim was not supported (§7.2, §7.3): at a matched budget a same-model summary was both more accurate and cheaper. We therefore position this paper not as the successful proposal of a new compressor, but as a *mixed-results study* that experimentally delimits where prompt-only symbolic compression fails and what its remaining value might be.

Findings preview. (1) ISM’s internal structure is used—inverting dictionary conclusions and removing symbolic structure each significantly lower accuracy (RQ1, dev scale). (2) At a matched budget Model Summary significantly beats ISM on accuracy and efficiency (RQ3). (3) On the reuse cost–accuracy frontier Model Summary again dominates ISM (RQ4). (4) The initial $\Delta_{\text{map}} \approx 0$ null was a measurement artifact (low compression purity + content-preserving corruption); once corrected, the RQ1 signal emerges. Taken together, ISM’s potential value is not “better compression” but a *checkable, transformable, executable semantic IR* handled by programs outside the LLM—beyond the scope of this prompt-only study.

2 Related Work

Prompt and context compression. Hard prompt compression selects task-important text. Selective Context removes low-information lexical units [10]; LLMingua uses a small LM for coarse-to-fine compression [7]; LongLLMingua adds question-aware relevance [8]; LLMingua-2 distills a task-agnostic compressor [9]. These are the most direct comparators for our fixed-budget experiment. ISM instead re-expresses rules and relations as a symbol–dictionary structure, enabling removal and swap of specific meaning units.

Continuous context compression. Gist Tokens [11], AutoCompressors [12], and ICAE [13] compress context into continuous memory; PCC and xRAG push high-ratio compression with learned or retrieval embeddings [14, 15]. Continuous memory has high capacity but its slots’ individual meanings are hard to inspect or swap; ISM trades capacity for explicit tokens and a dictionary.

Concept bottlenecks and intervenable representations. Concept Bottleneck Models place a human-defined concept layer before the prediction [1], extended by Concept Embedding Models [2], generative CBMs [3], and open-vocabulary CBMs [4]; Discover-then-Name discovers then names features [5], and Concept Bottleneck LLMs apply this to language models [6]. ISM connects to this line by intervening on an intermediate representation, but its primary goal is information preservation under a token budget, not concept supervision.

3 Inspectable Symbolic Compression

Problem. Given a document x , question q , and answer y , a compressor C maps x to a representation $z = C(x; B)$ within a token budget B , and a reasoner R produces $\hat{y} = R(z, q)$. ISM’s representation is $z_{\text{ISM}} = (S, D, G)$ with symbols $S = \{Z_1, \dots, Z_k\}$, a dictionary $D : Z_i \mapsto c_i$ mapping each symbol to a short textual definition (condition, event, exception, or relation), and relations G encoding application order, implication, exception, or conclusion.

Format and procedure. ISM uses a [DICTIONARY] block of **LABEL := definition** lines and a [RELATIONS] block. The compressor (a fixed instruction-tuned LLM) is prompted to emit only symbols, a short dictionary, and relations; to preserve conditions, conclusions, exceptions, priorities, thresholds, and ordering; not to answer any specific question; and to

keep the whole output within B whitespace tokens, regenerating rather than truncating on overflow. Each document is compressed once and reused for all of its questions.

Intervention operators. On a normal representation $z = (S, D, G)$ we define: *dictionary removal* $(S, \emptyset, G)$; *derangement* $(S, \pi(D), G)$, a label permutation of the definitions; *conclusion flip*, a counterfactual that inverts outcome tokens ($\text{HIGH} \leftrightarrow \text{LOW}$, $\text{True} \leftrightarrow \text{False}$) while keeping conditions and relations; *blanking*, replacing definitions with a placeholder; *random symbol control* matching length but removing meaning; and *label swap* to an unseen label set. The flip is the content-altering corruption introduced after the diagnostic in §6.

4 Research Questions

RQ1. Are the symbols and dictionary actually used in reasoning? **RQ2.** Can ISM use new label–meaning bindings beyond memorizing specific label strings? **RQ3.** At a matched token budget, does ISM preserve more task-relevant information than natural-language summaries and prior prompt compression? **RQ4.** How do cost and accuracy behave when one compressed representation is reused across many questions?

5 Experimental Setup

Data. Synthetic Rule-QA generates documents from a known gold rule graph (conjunction, disjunction, exception, precedence, threshold, temporal rules) with multiple questions whose gold answers are computed by a deterministic rule executor; the gold graph supports causal error analysis. For meaningful fixed-budget compression we use a *long-context profile*: each document is padded with deterministic, neutral filler to 700–2000 whitespace tokens *without* changing the rule graph, questions, or answers, so budgets of 64–512 are real compression. QASPER provides external validity on real papers; its answers are scored with token-level answer-F1 (§5), and it is excluded from reuse analysis.

Model and scoring. Compressor and reasoner are the same 4-bit Qwen2.5-7B-Instruct with greedy decoding ($T = 0$), seed 42, on a single T4 GPU. Synthetic answers are short labels scored by normalized exact match; QASPER uses token-level answer-F1 (SQuAD normalization, max over references, with unanswerable handling). We report Accuracy, Accuracy Retention $\text{AR} = \text{Acc}_m / \text{Acc}_{\text{full}}$, Compression Ratio $\text{CR} = \text{tok}_m / \text{tok}_{\text{full}}$, and Efficiency Score $\text{ES} = \text{AR} / \text{CR}$. Differences are tested on identical document–question pairs with McNemar’s exact test; 95% CIs use 10,000-sample paired bootstrap.

Scope of this study. All experiments are prompt-only at *dev* scale (development-set pilots, not the full registered scale of 5,000 dev documents). Tables labeled “dev pilot” use small N and are not cited as registered-scale numbers. The LoRA-based label-swap experiment (RQ2) and QASPER end-to-end runs are future work.

6 Experiments and the Diagnosis of an Early Null

We run three experiments: **6.1 Dictionary Ablation** (RQ1), **6.3 Fixed-Budget Comparison** (RQ3), and **6.4 Reuse** (RQ4).

An initial ablation produced $\Delta_{\text{map}} \approx 0$: corrupting the dictionary did not reduce accuracy. A diagnostic (**compress-audit**) attributed this to two measurement defects rather than to the hypothesis. **(i) Low purity.** The first LLM compressor frequently kept rule *conditions* but dropped their *conclusions* (rule coverage 0.70, self-containment 0.71); the representation often lacked the answer mapping, so the reasoner regressed toward the majority class. **(ii) Content-preserving corruption.** The standard derangement permutes which label holds which definition but preserves the *multiset* of definitions; with self-contained rules and arbitrary labels, the derivable answer is unchanged. We corrected (i) by requiring conclusion-bearing definitions (purity and self-containment $\rightarrow 1.0$; raising the compressor’s decoding budget to clear a truncation-induced **empty_relations** failure), and (ii) by adding

Table 1: Dictionary Ablation, dev scale-up ($N=40$ docs / 240 questions), LLM compressor + content-altering corruption. Contrasts: $\Delta_{\text{map}}^{\text{derange}}$ (label binding) = +0.071, CI $[-0.021, 0.163]$, $p=0.159$; $\Delta_{\text{map}}^{\text{flip}}$ (semantic content, primary) = +0.079, CI $[0.013, 0.146]$, $p=0.032$; $\Delta_{\text{symbol}} = +0.108$, CI $[0.050, 0.167]$, $p=5 \times 10^{-4}$.

Condition	Accuracy	AR	CR
Full Context	0.750	1.000	1.000
Full Symbol + Dict	0.446	0.594	0.745
Corrupted (derange)	0.375	0.500	0.745
Flipped (content)	0.367	0.489	0.745
Blank Dict	0.250	0.333	0.215
Symbol Only	0.413	0.550	0.079
Random Symbol	0.304	0.406	0.738

Table 2: Fixed-Budget dev pilot ($N=80$ questions). Cell entries are AR / ES. Full Context accuracy 0.700. LLM-Lingua-2 omitted (future baseline). ISM@64 not produced (representation floor).

Method	128	256	512
Model Summary	1.036 / 51.3	1.125 / 17.3	1.196 / 13.3
ISM	0.679 / 9.4	0.679 / 9.3	0.661 / 8.9
Keyword Extract	0.714 / 7.4	0.732 / 4.2	0.750 / 3.6
Oracle Gold Summary	0.732 / 11.0	0.732 / 11.0	0.732 / 11.0

the *conclusion-flip* content-altering corruption. Only after both corrections does the RQ1 signal appear.

7 Results (dev-scale)

7.1 Dictionary Ablation (RQ1)

Table 1 reports the corrected ablation with an LLM compressor and content-altering corruption ($N = 240$ questions). Permuting labels has no effect ($\Delta_{\text{map}}^{\text{derange}}$ not significant), but inverting conclusions and removing symbolic structure both lower accuracy significantly.

The dictionary’s *semantic content* and the *symbolic structure* are used; the arbitrary label binding is not. This is consistent across an LLM-produced ISM and a gold-graph-derived ISM. The amended pre-registered criterion “Symbol Only > Random *and* Full+Dict > Flipped” is met at dev scale.

7.2 Fixed-Budget Comparison (RQ3)

Table 2 compares methods at budgets $\{128, 256, 512\}$ on long-context documents ($N = 40$ docs / 80 questions; budget 64 is excluded as an ISM representation floor). Because AR is relative to Full Context, which is degraded by filler (§8), the *primary* comparison is the same-question paired contrast Model Summary vs. ISM, which is independent of that artifact.

Model Summary significantly outperforms ISM at every budget (McNemar $p < 0.01$, and $p < 10^{-6}$ at 512), and the gap grows with budget. Since both methods strip filler, this is independent of the filler artifact. The pre-registered criterion that ISM beat Model Summary in AR or ES is therefore *not met*.

7.3 Reuse cost–accuracy (RQ4)

Reusing one compression across n questions has analytic token cost (per document): $T_{\text{full}}(n) = n(|x| + |q|)$ and $T_{\text{cache}}(n) = |x| + |z| + n(|z| + |q|)$ (end-to-end), or $n(|z| + |q|)$ (serving-only). With $|x|=1348$, $|q|=5.5$ and per-method $|z|$, accuracy from the budget-256 fixed-budget run, Table 4 gives end-to-end token cost.

Table 3: Primary paired contrast Model Summary – ISM on identical questions ($N=80$). Discordant = (summary correct, ISM wrong) / (ISM correct, summary wrong).

Budget	Δ	95% CI	McNemar p	discordant
128	+0.250	[0.100, 0.400]	0.0029	31 / 11
256	+0.3125	[0.175, 0.450]	1.1×10^{-4}	33 / 8
512	+0.375	[0.250, 0.500]	2.3×10^{-7}	33 / 3

Table 4: Reuse end-to-end token cost vs. number of questions n (budget 256). All cached methods beat Full Context from $n=2$; among caches, Model Summary dominates ISM on both cost and accuracy.

Method	$n=1$	$n=8$	$n=32$	$n=64$	Accuracy
Full Context	1354	10828	43312	86624	0.700
Model Summary	1526	2168	4369	7305	0.787
ISM	1548	2265	4725	8005	0.475
Oracle Gold	1530	2184	4428	7420	0.512
Keyword	1820	3492	9225	16868	0.512

Reuse is a clear practical benefit of compression—every cached method beats Full Context from two questions and is $\sim 12\times$ cheaper at $n=64$ —but it is a *generic* property of any cacheable representation, not ISM-specific. On the cost–accuracy frontier, Model Summary dominates ISM (fewer tokens, higher accuracy).

8 Discussion

Efficiency vs. inspectability. RQ3 and RQ4 jointly close the “ISM is a better compressor / cache” route: a same-model summary is shorter and more accurate. Both ISM and summary are text produced and re-read by an LLM, so they fall in the same category, and within it the natural-language summary is the stronger baseline.

The filler artifact. In the fixed-budget run, Model Summary’s AR sometimes exceeded 1 (beating Full Context). A sanity check varying filler length (same documents) showed the summary–minus–full gap grows monotonically with filler ($-0.05 \rightarrow +0.15 \rightarrow +0.25$): with no filler, the summary does not beat full context. So $AR > 1$ is substantially a filler artifact (filler degrades full-context reasoning), which is why our primary RQ3 contrast is the filler-independent paired summary-vs-ISM comparison.

Manual edits are not the differentiator. “Summaries cannot be edited” is false—a human can edit a summary sentence. The honest distinction is that ISM, as a schema, admits *automatic*, *local*, *verifiable* edits and, in principle, symbolic execution without the LLM; a free-text summary does not. Demonstrating this is the subject of future work (Appendix B), not of the present prompt-only setup.

9 Limitations

Synthetic Rule-QA is more closed than real text; QASPER end-to-end evaluation is future work. Current RQ1 evidence is a dev scale-up ($N=240$), not the registered 5,000-document dev set; full-scale confirmation is deferred pending compression throughput improvements (compression is sequential per document and dominates cost). ES is a ratio that inflates at very small CR, so we always report AR and CR alongside it. System cost depends on more than token count (KV-cache, hardware, batch size, latency); the reuse model is analytic.

10 Conclusion

We studied ISM and compared it honestly to a same-model summary. The results are mixed. *What worked:* after diagnosis and correction, ISM’s symbolic structure and dictionary content are functionally used (RQ1, dev scale)—flipping conclusions and removing structure both significantly lower accuracy, while label permutation does not. *What did not:* the efficiency hypothesis is not supported—at a matched budget and under reuse, a natural-language summary is both more accurate and cheaper (RQ3, RQ4), so the pre-registered criterion that ISM beat Model Summary fails. Prompt-only symbolic compression is thus not a better compressor; its remaining promise is to be treated as a *programmatically executable semantic IR* handled by a parser/checker/executor outside the LLM (Appendix B), which we leave to future work along with RQ2 and full-scale runs.

Reproducibility

Dev-scale results, with raw artifacts or SHA-256 hashes, environment (commit/GPU/package versions), and reproduction commands, are organized per run under `docs/evidence/` in the code repository: 6.1 ablation (`ablation-qwen7b-N120`), 6.3 fixed-budget (`fixed-budget-N40`), 6.4 reuse (`reuse-N40`), and the filler sanity (`fixed-budget-filler-sanity`). Diagnosis and decisions are recorded under `docs/reviews/` and `docs/decisions/`. Large prediction files are kept in the GPU environment with recorded hashes and are regenerable from the stated commit and config.

A Pre-registered Criteria and Status

The minimum conditions registered before seeing dev results were: (1) ISM shows significantly higher AR or ES than Model Summary; (2) Symbol Only > Random *and* Full+Dict > Corrupted; (3) Unseen-Swap+Dictionary retains most of Original+Dictionary performance.

Amendment (post-diagnosis). The original derangement corruption preserves the multiset of definitions and tests surface label binding, not semantic dependence. The amended primary dictionary-dependence contrast is the semantic conclusion flip; derangement is reported as a secondary label-binding diagnostic. We report three contrasts: $\Delta_{\text{map}}^{\text{derange}}$ (label binding), $\Delta_{\text{map}}^{\text{flip}}$ (semantic content, primary), and Δ_{symbol} (symbolic structure).

Status (dev scale).

- Criterion #1 (ISM > Model Summary): **FAILED** — paired Summary–ISM = +0.25 to +0.375, McNemar $p < 0.01$ at all budgets (Table 3); reuse frontier also dominated by summary.
- Criterion #2 (Symbol Only > Random *and* Full+Dict > Flipped): **met** at dev scale ($\Delta_{\text{symbol}} p=5 \times 10^{-4}$, $\Delta_{\text{map}}^{\text{flip}} p=0.032$).
- Criterion #3 (label-swap transfer): **untested** — RQ2 future work.

Following the registration rule, because #1 failed we do not claim ISM’s efficiency advantage. #2 supports that ISM’s internal structure is used, which is a separate statement from efficiency.

B Future Work: Executable Semantic IR

Our negative/mixed results point to the main next direction. Today ISM is text an LLM generates and re-reads, so it shares a category with summaries, in which summaries are shorter and more accurate. For ISM to differ in kind, a program outside the LLM must handle it.

	Model Summary	ISM as semantic IR
Generation	LLM	LLM compiles to IR
Format	free text	schema (parsable)
Validation	hard	validator checks rules
Edit	manual rewrite	rule-id local, verifiable
Execution	needs LLM	symbolic executor, no LLM

Concretely: (1) *compile-and-execute*—parse the LLM-generated ISM to an AST and answer with a symbolic executor under the gold rule-graph semantics, without LLM QA; summaries lack an executor, so the categories finally separate. (2) *programmatic editability*—repair a corrupted/flipped ISM by a deterministic rule-id edit and measure `repair_success`, `targeted_effect_rate`, `collateral_damage_rate`, `parse_success`, `tokens_changed`, and `needs_llm_rewrite`, contrasting edit *locality* and *verifiability* with a summary’s manual rewrite. (3) RQ2 label swap (small LoRA), full registered-scale runs, and QASPER answer-F1.

References

- [1] P. W. Koh et al. Concept Bottleneck Models. *ICML*, 2020.
- [2] M. Espinosa Zarlenga et al. Concept Embedding Models. *NeurIPS*, 2022.
- [3] A. A. Ismail et al. Concept Bottleneck Generative Models. *ICLR*, 2024.
- [4] A. Tan et al. Concept Bottleneck Model with Open Vocabulary Concepts. *ECCV*, 2024.
- [5] S. Rao et al. Discover-then-Name: Task-Agnostic Concept Bottlenecks. *ECCV*, 2024.
- [6] C.-E. Sun et al. Concept Bottleneck Large Language Models. 2024.
- [7] H. Jiang et al. LLMingua: Compressing Prompts for Accelerated Inference. *EMNLP*, 2023.
- [8] H. Jiang et al. LongLLMingua. *ACL*, 2024.
- [9] Z. Pan et al. LLMingua-2. *Findings of ACL*, 2024.
- [10] Y. Li et al. Compressing Context to Enhance Inference Efficiency. *EMNLP*, 2023.
- [11] J. Mu, X. L. Li, N. Goodman. Learning to Compress Prompts with Gist Tokens. *NeurIPS*, 2023.
- [12] A. Chevalier et al. Adapting Language Models to Compress Long Contexts. *EMNLP*, 2023.
- [13] T. Ge et al. In-context Autoencoder for Context Compression. *ICLR*, 2024.
- [14] Y. Dai et al. Pretraining Context Compressor with Embedding-Based Memory. *ACL*, 2025.
- [15] X. Cheng et al. xRAG: Extreme Context Compression for RAG with One Token. *NeurIPS*, 2024.